

```

import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px

# -----
# PAGE CONFIGURATION
# -----
st.set_page_config(
    page_title="Rainfall-Surface Water Analysis",
    page_icon="🌧️",
    layout="wide"
)
# -----
# TITLE
# -----
st.title("🌧️ Rainfall-Surface Water Analysis")
st.markdown(
    """
    **GIS and Remote Sensing Dashboard**
    Analyze the relationship between rainfall and surface-water
    availability and identify drought-sensitive water bodies.
    """
)
# -----
# SAMPLE DATA
# -----
sample_data = pd.DataFrame({
    "Date": pd.date_range("2024-01-01", periods=12, freq="MS"),
    "Rainfall_mm": [
        45, 52, 70, 85, 110, 145,
        180, 160, 120, 90, 60, 40
    ],
    "Water_Area_km2": [
        1.20, 1.25, 1.35, 1.48, 1.70, 2.05,
        2.40, 2.30, 2.00, 1.72, 1.45, 1.25
    ]
})
sample_data["Date"] = pd.to_datetime(sample_data["Date"])
# -----
# SIDEBAR
# -----
st.sidebar.header("🌧️ Dashboard Controls")
uploaded_file = st.sidebar.file_uploader(
    "Upload rainfall & water-area CSV",
    type=["csv"]
)
if uploaded_file is not None:
    try:
        data = pd.read_csv(uploaded_file)
        data["Date"] = pd.to_datetime(data["Date"])
    except Exception:
        st.error("Unable to read the uploaded CSV file.")
    data = sample_data.copy()
else:
    data = sample_data.copy()
# -----
# DATA VALIDATION
# -----
required_columns = [
    "Date",
    "Rainfall_mm",
    "Water_Area_km2"
]
missing_columns = [
    col for col in required_columns
    if col not in data.columns
]
if missing_columns:
    st.error(
        "Missing columns: " +
        ", ".join(missing_columns)
    )
st.stop()
# -----
# CALCULATIONS
# -----
data = data.sort_values("Date")
total_rainfall = data["Rainfall_mm"].sum()
average_rainfall = data["Rainfall_mm"].mean()
maximum_water = data["Water_Area_km2"].max()
minimum_water = data["Water_Area_km2"].min()
water_loss_percent = (
    (maximum_water - minimum_water)
    / maximum_water
) * 100
correlation = data["Rainfall_mm"].corr(
    data["Water_Area_km2"]
)
# -----
# KPI CARDS
# -----
col1, col2, col3, col4 = st.columns(4)
with col1:
    st.metric(
        "Total Rainfall",
        f"{total_rainfall:.1f} mm"
    )
with col2:
    st.metric(
        "Average Rainfall",
        f"{average_rainfall:.1f} mm"
    )
with col3:
    st.metric(

```

```

"Maximum Water Area",
f"{maximum_water:.2f} km²"
)
with col4:
st.metric(
"Water Area Reduction",
f"{water_loss_percent:.1f}%"
)
st.divider()
# -----
# RAINFALL TREND
# -----
st.subheader("■ Rainfall Trend")
rainfall_fig = px.line(
data,
x="Date",
y="Rainfall_mm",
markers=True,
title="Monthly Rainfall"
)
rainfall_fig.update_layout(
xaxis_title="Date",
yaxis_title="Rainfall (mm)"
)
st.plotly_chart(
rainfall_fig,
use_container_width=True
)
# -----
# WATER AREA TREND
# -----
st.subheader("■ Surface-Water Area Trend")
water_fig = px.line(
data,
x="Date",
y="Water_Area_km2",
markers=True,
title="Surface-Water Area Over Time"
)
water_fig.update_layout(
xaxis_title="Date",
yaxis_title="Water Area (km²)"
)
st.plotly_chart(
water_fig,
use_container_width=True
)
# -----
# RAINFALL VS WATER AREA
# -----
st.subheader("■ Rainfall vs Surface-Water Area")
scatter_fig = px.scatter(
data,
x="Rainfall_mm",
y="Water_Area_km2",
trendline="ols",
hover_data=["Date"],
title="Relationship Between Rainfall and Water Area"
)
scatter_fig.update_layout(
xaxis_title="Rainfall (mm)",
yaxis_title="Water Area (km²)"
)
st.plotly_chart(
scatter_fig,
use_container_width=True
)
# -----
# CORRELATION
# -----
st.subheader("■ Rainfall-Water Correlation")
if correlation >= 0.7:
correlation_status = "Strong positive relationship"
elif correlation >= 0.3:
correlation_status = "Moderate positive relationship"
elif correlation >= -0.3:
correlation_status = "Weak relationship"
elif correlation >= -0.7:
correlation_status = "Moderate negative relationship"
else:
correlation_status = "Strong negative relationship"
col1, col2 = st.columns(2)
with col1:
st.metric(
"Correlation Coefficient",
f"{correlation:.3f}"
)
with col2:
st.info(correlation_status)
# -----
# VULNERABILITY ANALYSIS
# -----
st.subheader("■ Water-Body Vulnerability Analysis")
water_reduction = water_loss_percent
if water_reduction >= 50:
vulnerability = "HIGH"
elif water_reduction >= 25:
vulnerability = "MEDIUM"
else:
vulnerability = "LOW"
st.write(
f"Based on the observed water-area reduction, "
f"the current vulnerability level is **{vulnerability}**."
)

```

```

# -----
# MONTHLY CHANGE
# -----
data["Water_Change_%"] = (
    data["Water_Area_km2"]
    .pct_change() * 100
)
change_fig = px.bar(
    data,
    x="Date",
    y="Water_Change_%",
    title="Monthly Surface-Water Change"
)
change_fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Water Area Change (%)"
)
st.plotly_chart(
    change_fig,
    use_container_width=True
)
# -----
# DATA TABLE
# -----
st.subheader("■ Analysis Data")
display_data = data.copy()
display_data["Date"] = (
    display_data["Date"]
    .dt.strftime("%Y-%m-%d")
)
st.dataframe(
    display_data,
    use_container_width=True
)
# -----
# DOWNLOAD RESULTS
# -----
csv_data = data.to_csv(index=False)
st.download_button(
    label="■ Download Analysis CSV",
    data=csv_data,
    file_name="rainfall_surface_water_analysis.csv",
    mime="text/csv"
)
# -----
# FOOTER
# -----
st.divider()
st.caption(
    "Rainfall-Surface Water Response Analysis | "
    "GIS & Remote Sensing Project"
)

```

How to Run
 Save the code as app.py.
 Open Command Prompt or Terminal in the project folder.
 Install the required packages using the command above.
 Run the application with: streamlit run app.py.
 Open the local URL shown by Streamlit in your browser.